

# Análise de Desempenho do Problema de N-Corpos: Um estudo comparativo entre Implementações Sequencial e Paralela em GPU (HIP)

Geovanna Campos Pires<sup>1</sup>

<sup>1</sup>Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)  
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

geovanna.pires@inf.ufrgs.br

**Abstract.** *The behavior of a system composed of more than two bodies cannot be described using closed-form equations. Known as the N-body problem, this challenge in Physics has driven the development of computational solutions to simulate the evolution of such systems. However, sequential implementations exhibit poor performance due to their quadratic computational complexity as the number of bodies increases. In this context, parallelization emerges as an alternative to reduce execution time. Therefore, this work analyzes the performance of sequential (CPU) and parallel (GPU/HIP) implementations, comparing their results using metrics such as speedup to identify the most advantageous approach under different scenarios.*

**Resumo.** *O comportamento de um sistema com mais de dois corpos não pode ser descrito por fórmulas fechadas. Conhecido como o problema de N-corpos, esse desafio da Física desencadeou o desenvolvimento de soluções computacionais para simular a evolução desses sistemas. Contudo, abordagens sequenciais apresentam baixo desempenho devido à complexidade quadrática, à medida que o número de corpos aumenta. Dessa forma, a paralelização surge como uma alternativa para reduzir o tempo de execução. Nesse cenário, o presente trabalho buscou analisar a atuação de implementações sequencial (CPU) e paralela (GPU/HIP), comparando seus resultados, utilizando métricas como o speedup, para entender a alternativa mais vantajosa em diferentes cenários.*

## 1. Introdução

A Mecânica Clássica, estabeleceu as bases para a análise e compreensão do movimento de corpos sob a ação de forças. Destacam-se, nessa área, as propostas do matemático e físico Isaac Newton, em especial a Lei Universal da Gravitação de 1687, segundo a qual dois corpos exercem entre si uma força gravitacional proporcional ao produto de suas massas e inversamente proporcional ao quadrado da distância entre eles [Newton 1687].

Definida pela expressão:

$$F = G \frac{m_1 m_2}{r^2}$$

A partir dessa formulação, foi possível descrever e prever trajetórias de sistemas simples. Entretanto, esse cenário muda quando o número de corpos é maior do que dois

( $N > 2$ ), pois não existe uma fórmula fechada capaz de descrever seu comportamento [Sandoval 2024]. Conhecido como o problema de N-corpos, esse desafio está presente em inúmeras aplicações científicas.

Devido a essa limitação, buscaram-se alternativas, levando ao desenvolvimento de métodos numéricos implementados por meio de simulações computacionais. Em seu método mais direto, conhecido como "*força bruta*", calcula-se a interação gravitacional entre todos os pares de corpos em cada passo de tempo, sendo essa abordagem implementada, principalmente, por meio de algoritmos sequenciais. Porém, ao longo do tempo, notou-se um impasse: a impraticabilidade da solução sequencial para grandes valores de  $N$ . Devido à sua complexidade computacional quadrática,  $O(N^2)$ , o tempo de execução torna-se exorbitante, já que, com o aumento do número de corpos, o número de cálculos cresce quadraticamente.

É nesse cenário que a Computação de Alto Desempenho surge como uma alternativa. Por meio da paralelização e do uso de arquiteturas como as **Graphics Processing Units (GPUs)**, torna-se possível reduzir o tempo de execução por meio da realização simultânea de cálculos.

Apesar de sua promissora proposta, mesmo a implementação paralela pode nem sempre ser a que entrega a melhor solução. Gargalos podem surgir em determinados cenários, decorrentes de fatores como o acesso intensivo à memória e a manipulação de um grande volume de dados, limitando os ganhos de desempenho.

Diante desse cenário, o presente trabalho busca entender em quais situações, por meio de resultados experimentais, a CPU deixa de ser uma opção viável. Para isso, foram implementadas e avaliadas duas versões da simulação: uma sequencial, executada em CPU, e outra paralela, executada em GPU. A análise busca avaliar o ganho de desempenho por meio do *speedup*, bem como compreender os fatores que influenciam o desempenho da implementação em GPU, em especial o impacto de diferentes tamanhos de bloco na execução, identificando em quais situações a execução em GPU supera, ou não, a CPU.

Para além da introdução, esse artigo está organizado da seguinte forma: a Seção 2 discute componentes conceituais. Na Seção 3 é apresentado o trabalho utilizado de base para o desenvolvimento da pesquisa. A Seção 4 detalha o ambiente experimental e as implementações. A Seção 5 analisa os resultados obtidos. Por fim, a Seção 6 conclui o artigo.

## 2. Fundamentação Teórica

### 2.1. O Problema de N-Corpos

A partir da Lei da Gravidade de Newton, é possível descrever o comportamento de sistemas simples, como o Terra-Lua, no qual os corpos orbitam em torno de um centro de massa comum. Sistemas de até dois corpos são problemas que possuem soluções definidas e podem ser resolvidos analiticamente, ou seja, suas trajetórias podem ser determinadas diretamente por equações matemáticas [Sandoval 2024].

Um impasse surge quando o número de corpos aumenta. Conhecido como o problema de **N-corpos** [Babadzanjanjanz 1979], nesse cenário o movimento de cada corpo depende da interação simultânea com todos os demais. O sistema se torna não integrável

[Morales-Ruiz 1999], inviabilizando a descrição de suas trajetórias por meio de fórmulas fechadas, tornando-o significativamente mais complexo.

Dessa forma, a evolução do sistema é calculada de maneira numérica, o cálculo das forças gravitacionais é realizado a cada interação de partículas, e a força resultante é proveniente da soma de cada interação. Em busca de uma alternativa eficiente para realizar esses cálculos, físicos e cientistas recorrem a simulações computacionais para analisar a evolução desses sistemas ao longo do tempo.

Logo, em um sistema de  $N$  corpos, cada um realiza o cálculo de interação com  $N - 1$  corpos. Assim, o número total de interações cresce proporcionalmente a  $N(N - 1)$ , computacionalmente, essa complexidade é definida como  $O(N^2)$ .

## 2.2. Paralelismo

Apesar de ser possível implementar algoritmos sequenciais para o cálculo das interações do problema de N-corpos, sua natureza quadrática inviabiliza a execução dessas implementações para um número elevado de corpos, frequentemente requerido em aplicações científicas.

Essas limitações tornam o problema um candidato natural à exploração de paralelismo. Computacionalmente, paralelismo refere-se à execução simultânea de diferentes tarefas que contribuem para a mesma aplicação [Gonnord et al. 2023]. Aplicações podem ser paralelas por natureza, ou seja, sua estrutura admite que diferentes partes sejam realizadas de maneira independente e simultânea.

Essa propriedade se aplica ao problema de N-corpos. O cálculo das forças exercidas sobre cada corpo pode ser realizado separadamente para cada entidade ao longo do mesmo passo de simulação, aumentando a eficiência da aplicação e permitindo a execução de simulações com grandes valores de entrada, adequando-se a cenários de experimentos científicos.

## 3. Trabalhos Relacionados

Por se tratar de um desafio fundamental para a Matemática e para a Física [Li and Liao 2025], o problema de N-corpos vem sendo explorado há anos e recebe inúmeras abordagens e até mesmo soluções analíticas para casos específicos.

Euler [Euler 1767], por meio das soluções colineares, trabalhou o cenário em que três corpos estão dispostos em uma linha reta, alinhados no espaço. Já Lagrange [Lagrange 1772] estudou três corpos que formam um triângulo equilátero, desenvolvendo as equações equilaterais.

Da mesma forma, inúmeras soluções algorítmicas surgiram no cenário computacional. Inicialmente, os códigos desenvolvidos apresentavam complexidade quadrática de  $O(N^2)$  como os propostos por [von Hoerner 1960] e [Aarseth 1971]. Entretanto, ao longo do tempo, buscando a otimização desses algoritmos, surgiram novas abordagens que conseguiram diminuir essa complexidade. [Barnes and Hut 1986] reduziram a complexidade para  $O(N \log N)$ , enquanto [Greengard and Rokhlin 1987] alcançou uma complexidade linear,  $O(N)$ , com o *Fast Multipole Method*.

Devido à natureza paralela do problema, ou seja, o fato de que o cálculo de forças exercidas sobre cada corpo poder ser realizado independentemente, e à sua complexidade

quadrática, o problema de N-corpos tornou-se um benchmark da Computação de Alto Desempenho. Dessa forma, inúmeros pesquisadores utilizaram as ideias desenvolvidas e realizaram melhorias e adaptações nas aplicações existentes para avaliar seu desempenho de diversas maneiras. [Thüring 2023], por exemplo, realiza uma comparação do algoritmo Barnes-Hut em ambientes de CPU e GPU para compreender o comportamento em tempo de execução dessas implementações. Já [Meurer 2020] realiza uma análise de uma implementação do algoritmo de Barnes-Hut em CUDA para avaliar o desempenho em ambientes Multi-GPU.

Embora existam diversas pesquisas no escopo do tema, neste trabalho optou-se por limitar a análise ao artigo *Study and analysis of parallel N-body simulations* [Arcencio and Guardia 2024]. Essa escolha está alinhada ao caráter educacional deste trabalho, desenvolvido no contexto de uma disciplina de mestrado, com restrições de prazo e ferramentas, permitindo uma melhor compreensão e comparação dos resultados. Além disso, ressalta-se a transparência das implementações e dos resultados da pesquisa, disponibilizados em um repositório *on-line* de acesso público, facilitando o desenvolvimento desta proposta.

Em [Arcencio and Guardia 2024], os autores apresentam um estudo comparativo entre duas implementações paralelas para o problema de N-corpos. Abordando arquiteturas multicore e GPUs, eles analisam as diferenças de resultados e avaliam a influência de cada método no tempo de execução e escalabilidade. Para o cálculo da movimentação dos corpos, foi utilizada a integração de Verlet, que atualiza a posição de uma partícula por meio de suas duas posições anteriores e de seu vetor de aceleração atual.

O ambiente experimental consistiu em duas máquinas: uma equipada com um Intel Xeon Silver 4208, com 16 núcleos físicos e 32 núcleos lógicos, utilizada para a implementação multicore, e outra com uma NVIDIA GeForce RTX 2080 Ti para a implementação em GPU. Foram realizadas cinco execuções de dez passos para três entradas ( $N$ ) diferentes: 25.000, 50.000 e 100.000 corpos. As especificações do código e dos experimentos podem ser conferidas no repositório GitHub dos pesquisadores <sup>1</sup>.

Como conclusão, os autores estabeleceram que, embora a implementação multicore aumente a eficiência da simulação de N-corpos, a GPU é capaz de alcançar resultados ainda mais expressivos, entregando tempos de execução significativamente menores.

## 4. Metodologia

Todos os códigos, os procedimentos realizados nos experimentos e os resultados obtidos foram disponibilizados em um repositório no GitHub<sup>2</sup>, com o objetivo de tornar a pesquisa reprodutível e contribuir para uma melhor compreensão por parte dos leitores.

### 4.1. Ambiente Experimental

No que se refere aos ambientes experimentais, as execuções sequenciais foram realizadas em um computador *desktop*, cujas especificações podem ser conferidas na Tabela 1

Buscando também contribuir com o estudo de [Arcencio and Guardia 2024] sob uma perspectiva diferente, optou-se por utilizar uma arquitetura de GPU diferente, baseada em AMD. Dessa forma, para as execuções paralelas, foram utilizadas duas partições

---

<sup>1</sup><https://github.com/GuiArcencio/parallel-nbody>

<sup>2</sup><https://github.com/geovannapires/CMP270-HPC-UFRGS>

pertencentes ao Parque Computacional de Alto Desempenho (PCAD) da Universidade Federal do Rio Grande do Sul (UFRGS), denominadas Lunarís e Sirius, cujas características arquiteturais estão apresentadas na Tabela 2.

**Tabela 1. Ambiente experimental da versão sequencial.**

| Componente        | Especificação                      |
|-------------------|------------------------------------|
| Processador       | Intel Core i7-12700F (12ª geração) |
| Arquitetura       | x86_64                             |
| Núcleos físicos   | 12                                 |
| Threads           | 20                                 |
| Frequência máxima | 4,9 GHz                            |

**Tabela 2. Ambientes experimentais utilizados nos experimentos em GPU.**

| Componente        | Lunarís                           | Sirius                |
|-------------------|-----------------------------------|-----------------------|
| Processador       | AMD Ryzen Threadripper PRO 5965WX | AMD Ryzen 9 3950X     |
| Arquitetura       | x86_64                            | x86_64                |
| Núcleos físicos   | 24                                | 16                    |
| Threads           | 48                                | 32                    |
| Frequência máxima | 7,02 GHz                          | 4,76 GHz              |
| GPU               | 2 × AMD Radeon RX 7900 XT         | AMD Radeon RX 7900 XT |
| Memória da GPU    | 20 GB (cada GPU)                  | 20 GB                 |
| Compute Units     | 84                                | 84                    |

**4.2. Implementação**

Para garantir uma comparação consistente entre as duas aplicações, foram estabelecidos parâmetros de execução idênticos para ambas as abordagens, sendo eles:

- Entradas com valores de: 1024, 2048, 4096, 25k, 50k e 100k;
- Número de repetições: 5;
- Número de passos: 10;
- *Seed*: 42;

Ademais, para a implementação em GPU, os testes também foram realizados utilizando diferentes tamanhos de bloco (64, 128, 256), com o objetivo de analisar o impacto de diferentes *Block Sizes* no desempenho da aplicação.

**4.2.1. Sequencial - CPU**

Para a realização dos testes em CPU, utilizou-se do código disponibilizado por [Arcencio and Guardia 2024]. Desenvolvido na linguagem C, o programa recebe os parâmetros necessários para a sua inicialização: a quantidade de partículas, o número de passos da simulação e um *seed*. Este último foi estabelecido para manter a reprodutibilidade dos experimentos, fixando o estado inicial do gerador de números aleatórios. Em seguida, os corpos são inicializados com uma posição, massa e velocidade.

A principal função desse algoritmo é a responsável pelo cálculo da aceleração gravitacional de cada partícula. É nela que o gargalo da aplicação sequencial se apresenta, devido à implementação direta do problema de N-corpos por meio de um *loop* aninhado. Dessa forma, a complexidade computacional da aplicação é definida como  $O(N^2)$ . Logo, o número de interações e o tempo de execução crescem quadraticamente à medida que o número de partículas aumenta.

Além disso, o tempo total de execução é obtido por meio da função *clock\_gettime*. Os valores, registrados em segundos, foram utilizados para a comparação de desempenho entre a CPU e as GPUs.

#### 4.2.2. Paralela - GPU

Na versão paralela, o código em CUDA elaborado por [Arcencio and Guardia 2024] foi utilizado como base para a elaboração de uma versão em AMD HIP. Embora algumas modificações tenham sido realizadas, manteve-se a mesma lógica de simulação, de modo que a comparação considerasse apenas os ganhos da paralelização.

Em especial, destacam-se as mudanças na função de cálculo da aceleração. Na versão paralela, o *loop* aninhado foi desfeito com a eliminação do laço externo, que deu lugar a um *kernel* HIP, no qual cada *thread* recebe uma partícula para processamento. Essa estratégia permite que o cálculo da aceleração seja realizado em paralelo, reduzindo o tempo de execução, porém, mantendo a complexidade quadrática. Por fim, assim como na versão sequencial, o tempo medido corresponde apenas ao período de execução da simulação.

### 5. Resultados

A partir dos dados obtidos nas execuções, diversas análises puderam ser realizadas. Neste trabalho, o foco concentrou-se na avaliação do comportamento individual de cada implementação, considerando seus respectivos *runtimes* e o comportamento esperado, bem como na comparação entre as versões por meio da métrica de *speedup*.

#### 5.1. CPU

A análise da implementação sequencial teve como objetivo verificar se a implementação seguiu a complexidade esperada para o problema de N-corpos. Os resultados indicam que a aplicação apresentou o comportamento esperado, com um aumento significativo do tempo de execução à medida que o número de partículas foi ampliado.

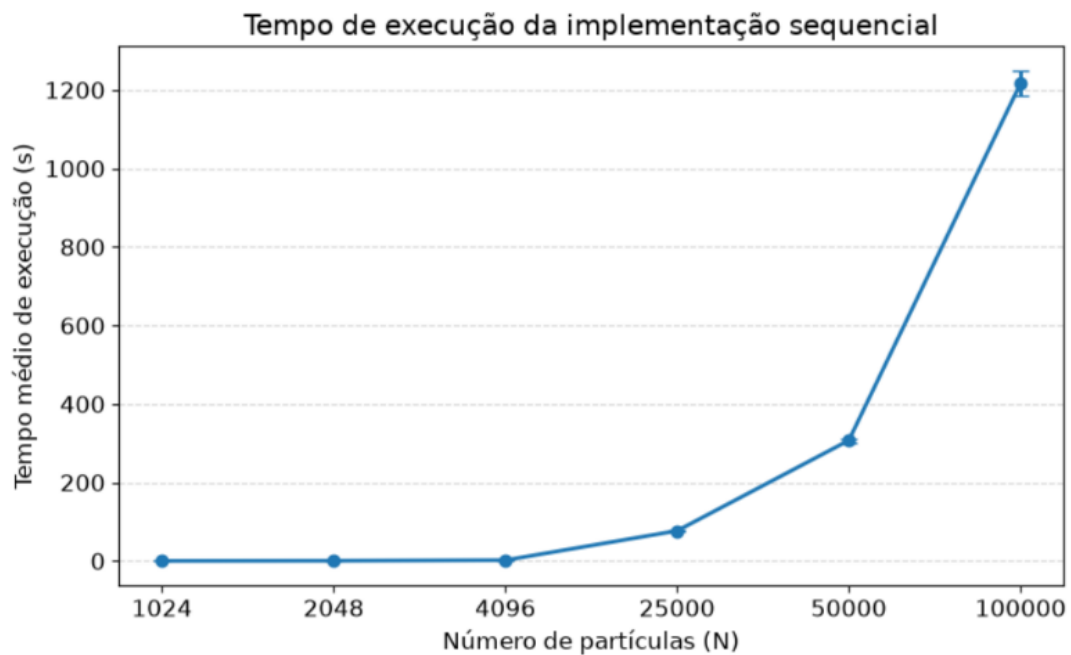
Essa característica foi confirmada pelos dados da Figura 1, que apresenta uma comparação da razão entre os tempos observados e a razão teórica esperada para um algoritmo de complexidade  $O(N^2)$ . Constata-se que os valores experimentais estão em concordância com os valores teóricos, com destaque para a persistência desse comportamento até mesmo para o salto de 4096 para 25000 partículas.

Além dos dados quantitativos, esse comportamento também pode ser compreendido visualmente por meio do gráfico da Figura 2.

Essa implementação estabelece um *baseline* para comparação com a versão paralela.

| N      | RAZAO_TEMPO | RAZAO_ESPERADA | ERRO (%) |
|--------|-------------|----------------|----------|
| 1024   | nan         | nan            | nan      |
| 2048   | 3.762333    | 4.000000       | 5.94     |
| 4096   | 3.831376    | 4.000000       | 4.22     |
| 25000  | 37.528959   | 37.252903      | 0.74     |
| 50000  | 3.996623    | 4.000000       | 0.08     |
| 100000 | 3.960290    | 4.000000       | 0.99     |

**Figura 1. Tabela de comparação de razões da CPU.**



**Figura 2. Gráfico de tempo de execução da implementação sequencial.**

## 5.2. GPUs

Em virtude da utilização de duas partições com arquiteturas diferentes nos experimentos de GPU, os resultados da Lunarix e da Sirius foram analisados separadamente. Apesar de ambas serem AMD e utilizarem o código em HIP, as diferenças de hardware podem levar a desempenhos distintos.

Nessa seção, o foco consiste no estudo dos *Block Sizes* e sua influência no desempenho. Os resultados completos de cada execução foram disponibilizados no repositório.

As Tabelas 3 e 4 expõem os melhores *Block Sizes* para cada tamanho de entrada em cada uma das partições. Nos dois casos, observou-se que não existe uma configuração que se destaque de forma consistente para todos os diferentes  $N$ . Porém, o *Block Size* 128 apresentou o menor tempo médio em 66,67% das execuções em ambas as arquiteturas.

Para mais, as diferenças entre os tempos médios obtidos para os *Block Sizes* foram

pequenas, não indicando sensibilidade significativa da aplicação a diferentes tamanhos de blocos.

**Tabela 3. Resultados da análise do tamanho de bloco (*block size*) na GPU Luno-  
ris.**

| Block Size | N      | Média (s) | Mínimo (s) | Máximo (s) | Mediana (s) |
|------------|--------|-----------|------------|------------|-------------|
| 128        | 1024   | 0.082532  | 0.080659   | 0.084353   | 0.082815    |
| 128        | 2048   | 0.085837  | 0.085014   | 0.087198   | 0.085909    |
| 128        | 4096   | 0.088379  | 0.085949   | 0.089637   | 0.089270    |
| 64         | 25000  | 0.103350  | 0.101388   | 0.105620   | 0.102746    |
| 256        | 50000  | 0.149763  | 0.148568   | 0.151174   | 0.149600    |
| 128        | 100000 | 0.307708  | 0.307099   | 0.308881   | 0.307464    |

**Tabela 4. Resultados da análise do tamanho de bloco (*block size*) na GPU Sirius.**

| Block Size | N      | Média (s) | Mínimo (s) | Máximo (s) | Mediana (s) |
|------------|--------|-----------|------------|------------|-------------|
| 128        | 1024   | 0.151487  | 0.147825   | 0.155074   | 0.150185    |
| 256        | 2048   | 0.153989  | 0.149599   | 0.156957   | 0.154148    |
| 128        | 4096   | 0.156335  | 0.153718   | 0.159290   | 0.155209    |
| 128        | 25000  | 0.171741  | 0.168852   | 0.174286   | 0.171737    |
| 64         | 50000  | 0.219189  | 0.217425   | 0.221504   | 0.219021    |
| 128        | 100000 | 0.400908  | 0.398774   | 0.404489   | 0.400370    |

### 5.3. CPU X GPU

Devido à utilização de três tamanhos de *Block Sizes* (64, 128, 256) diferentes nos testes de GPU, foi necessário definir uma configuração de referência para a comparação com a implementação sequencial. Assim, optou-se pela escolha daquele de melhor desempenho geral em ambas as partições, dessa forma, adotou-se o *Block Size* de 128.

A Tabela 5 apresenta os tempos médios obtidos pela implementação sequencial e pelas implementações em GPU utilizando o *Block Size* 128. Como pode ser observado, as GPUs apresentaram os menores tempos em praticamente todos os casos, com exceção da entrada de  $N = 1024$ , na qual a CPU teve um desempenho melhor do que a partição Sirius.

Já em relação ao cálculo de *speedup*, foi utilizada a fórmula:

$$S = \frac{T_{cpu}}{T_{gpu}}$$

em que:

- se  $S \geq 1$ , a GPU foi mais rápida que a CPU;
- se  $S = 1$ , GPU e CPU tiveram o mesmo desempenho;
- se  $S < 1$ , a GPU foi mais lenta que a CPU.



De acordo com os resultados apresentados na Tabela 6, para o menor valor de entrada,  $N = 1024$ , na Lunarix a GPU e a CPU obtiveram um mesmo desempenho. Já a Sirius, obteve um desempenho ligeiramente inferior ao da CPU. Todavia, a partir de  $N = 2048$ , ambas as GPUs passaram a superar a implementação sequencial. A partir disso, entende-se que, para valores muito pequenos, o custo das operações de alocação de memória e das transferências de dados podem se tornar um problema para o desempenho da GPU.

**Tabela 5. Tempo médio de execução (s) da implementação sequencial e das execuções em GPU.**

| N      | CPU         | LUNARIS  | SIRIUS   |
|--------|-------------|----------|----------|
| 1024   | 0.142090    | 0.082532 | 0.151487 |
| 2048   | 0.534591    | 0.085837 | 0.156702 |
| 4096   | 2.048221    | 0.088379 | 0.156335 |
| 25000  | 76.867595   | 0.104254 | 0.171741 |
| 50000  | 307.210828  | 0.153316 | 0.225287 |
| 100000 | 1216.643848 | 0.307708 | 0.400908 |

**Tabela 6. Speedup das implementações em GPU em relação à versão sequencial.**

| N      | Speedup LUNARIS | Speedup SIRIUS |
|--------|-----------------|----------------|
| 1024   | 1.721648        | 0.937970       |
| 2048   | 6.228012        | 3.411507       |
| 4096   | 23.175424       | 13.101519      |
| 25000  | 737.313578      | 447.579633     |
| 50000  | 2003.770164     | 1363.642057    |
| 100000 | 3953.895996     | 3034.720803    |

## 6. Conclusão

Este trabalho teve como objetivo a implementação sequencial em CPU e paralela em GPU (HIP) do problema de N-corpos, a fim de analisar e comparar seus desempenhos. De acordo com os resultados obtidos, conclui-se que a implementação sequencial apresentou o comportamento esperado de  $O(N^2)$ . Em relação à aplicação paralela constatou-se uma redução considerável do tempo de execução do problema. Quanto ao impacto de diferentes tamanhos de *Block Size*, não houve um efeito significativo no desempenho, embora o *Block Size* 128 tenha apresentado resultados mais consistentes. Ademais, a GPU mostrou-se a melhor escolha para cenários com um grande número de corpos, passando a compensar os custos para os valores utilizados neste estudo, a partir de 2048 partículas.

Quando comparado com o trabalho utilizado como base, nota-se uma equivalência nas constatações obtidas. Por fim, entende-se que as implementações estão em concordância com a previsão teórica e também com os resultados reportados na literatura, reforçando a eficiência da paralelização para a resolução do problema de N-corpos.

## Referências

- Aarseth, S. J. (1971). Direct Integration Methods of the N-Body Problem (Papers appear in the Proceedings of IAU Colloquium No. 10 Gravitational N-Body Problem (ed. by Myron Lecar), R. Reidel Publ. Co. , Dordrecht-Holland.). , 14(1):118–132.
- Arcencio, G. and Guardia, H. (2024). Study and analysis of parallel n-body simulations. In *Anais da XV Escola Regional de Alto Desempenho de São Paulo*, pages 5–8, Porto Alegre, RS, Brasil. SBC.
- Babadzanjan, L. K. (1979). Existence of the continuations in the N-body problem. *Celestial Mechanics*, 20(1):43–57.
- Barnes, J. and Hut, P. (1986). A hierarchical  $O(N \log N)$  force-calculation algorithm. , 324(6096):446–449.
- Euler, L. (1767). De motu rectilineo trium corporum se mutuo attrahentium. *Novi Commentarii Academiae Scientiarum Imperialis Petropolitanae*, 11:144–151. Eneström E327.
- Gonnord, L., Henrio, L., Morel, L., and Radanne, G. (2023). A survey on parallelism and determinism. *ACM Comput. Surv.*, 55(10).
- Greengard, L. and Rokhlin, V. (1987). A fast algorithm for particle simulations. *Journal of Computational Physics*, 73(2):325–348.
- Lagrange, J.-L. (1772). Recherches sur la libration de la lune, dans lesquelles on cherche si la lune, en tournant autour de la terre, tourne elle-même autour de son axe. *Recueil des Pièces qui ont remporté le Prix de l'Académie Royale des Sciences*, 9:1–50.
- Li, X. and Liao, S. (2025). A review on periodic orbits of the general planar three-body problem. *Science China Physics, Mechanics & Astronomy*, 68(8):289501.
- Meurer, B. B. (2020). Implementação paralela do algoritmo barnes-hut para simulação do problema n-corpos usando um número arbitrário de gpu's. <http://hdl.handle.net/11422/13430>. Orientador: Silvana Rossetto.
- Morales-Ruiz, J. (1999). *Differential Galois Theory and Non-Integrability of Hamiltonian Systems*.
- Newton, I. (1687). *Philosophiæ Naturalis Principia Mathematica*. Jussu Societatis Regiæ ac Typis Josephi Streater, London. Primeira Edição.
- Sandoval, F. (2024). The n-body problem.
- Thüring, T. (2023). *Comparison of different n-body algorithms on various hardware platforms using SYCL*. PhD thesis, University of Stuttgart.
- von Hoerner, S. (1960). Die numerische Integration des n-Körper-Problemes für Sternhaufen. I. , 50:184–214.